

1 Liniowa klasyfikacja binarna

Rozpatrzmy dwa abstrakcyjne zbiory danych różniących się ontologią, zakładając, że lokalizują się one w n -wymiarowej przestrzeni euklidesowej $\mathbb{R}^n$. W ogólności przestrzeń, której elementy poddawane są zadaniom uczenia maszynowego nazywamy *przestrzenią cech*. Rozpoczynamy od zdefiniowania wzorców klas - tutaj są to dwa pojedyncze punkty $u^{(1)}$, $u^{(2)}$ ('pojedynczość' wzorców klas umożliwi istnienie jednoznacznego rozwiązania analitycznego klasyfikacji). Klasy będziemy określać symbolami $C^{(1)}$ oraz $C^{(2)}$ i definiujemy je przez relację bliskości elementów względem wzorców $u^{(1)}$ i $u^{(2)}$, przyjmując jako miarę odległości metrykę euklidesową (poniżej symbol $\| \cdot \|$ oznacza normę euklidesową):

$$C^{(1)} = \left\{ x \in \mathbb{R}^n : \|x - u^{(1)}\| < \|x - u^{(2)}\| \right\}, \quad (1)$$

i analogicznie

$$C^{(2)} = \left\{ x \in \mathbb{R}^n : \|x - u^{(2)}\| < \|x - u^{(1)}\| \right\} \quad (2)$$

Traktując wektory jako macierze kolumnowe, uczciwa granica pomiędzy klasami $C^{(1)}$ i $C^{(2)}$ jest zbiorem punktów będących rozwiązaniem równania

$$\|x - u^{(1)}\| = \|x - u^{(2)}\| \quad (3)$$

$$(x - u^{(1)})^T (x - u^{(1)}) = (x - u^{(2)})^T (x - u^{(2)}) \quad (4)$$

Ponadto

$$\begin{aligned} x^T x - x^T u^{(1)} - u^{(1)T} x^{(1)} + u^{(1)T} u^{(1)} = \\ \|x\|^2 - \|u^{(1)}\|^2 - x^T u^{(1)} + u^{(1)T} x \end{aligned} \quad (5)$$

Stąd

$$\begin{aligned} \|x\|^2 + \|u^{(1)}\|^2 - x^T u^{(1)} - u^{(1)T} x &= \|x\|^2 + \|u^{(2)}\|^2 - x^T u^{(2)} - u^{(2)T} x \\ \|u^{(1)}\|^2 - \|u^{(2)}\|^2 &= x^T u^{(1)} - x^T u^{(2)} + u^{(1)T} x - u^{(2)T} x \\ \|u^{(1)}\|^2 - \|u^{(2)}\|^2 &= x^T (u^{(1)} - u^{(2)}) + (u^{(1)T} - u^{(2)T}) x \\ \|u^{(1)}\|^2 - \|u^{(2)}\|^2 &= x^T (u^{(1)} - u^{(2)}) + (u^{(1)} - u^{(2)})^T x \\ \|u^{(1)}\|^2 - \|u^{(2)}\|^2 &= [(u^{(1)} - u^{(2)})^T x]^T + (u^{(1)} - u^{(2)})^T x \\ \|u^{(1)}\|^2 - \|u^{(2)}\|^2 &= (u^{(1)} - u^{(2)})^T x + (u^{(1)} - u^{(2)})^T x \\ \|u^{(1)}\|^2 - \|u^{(2)}\|^2 &= 2(u^{(1)} - u^{(2)})^T x \\ \frac{1}{2} (\|u^{(1)}\|^2 - \|u^{(2)}\|^2) &= (u^{(1)} - u^{(2)})^T x \end{aligned} \quad (6)$$

Powyższe równanie jest równaniem hiperpłaszczyzny rozdzielającej elementy klas w przestrzeni $\mathbb{R}^n$. Równanie (6) jest równoważne następującej formule:

$$w^T x = \bar{w}_0, \quad (7)$$

gdzie

$$w^T = (u^{(1)} - u^{(2)})^T \quad (8)$$

$$\bar{w}_0 = \frac{1}{2} \left(\|u^{(1)}\|^2 - \|u^{(2)}\|^2 \right) \quad (9)$$

W przypadku $n = 2$ hiperpłaszczyzna jest prostą, gdyż

$$u^{(1)} = [u_x^{(1)}, u_y^{(1)}]^T \quad u^{(2)} = [u_x^{(2)}, u_y^{(2)}]^T$$

$$x = [x_1, x_2]^T =: [x, y]^T = \begin{bmatrix} x \\ y \end{bmatrix}$$

$$u^{(1)} - u^{(2)} = \begin{bmatrix} u_x^{(1)} - u_x^{(2)} \\ u_y^{(1)} - u_y^{(2)} \end{bmatrix}$$

$$w^T = (u^{(1)} - u^{(2)})^T = [u_x^{(1)} - u_x^{(2)}, u_y^{(1)} - u_y^{(2)}] =: [a, b]$$

Definiując

$$-c := \bar{w}_0 = w^T x = [a, b] \begin{bmatrix} x \\ y \end{bmatrix}$$

otrzymujemy klasyczne równanie prostej w $\mathbb{R}^2$

$$ax + by + c = 0 \quad (10)$$

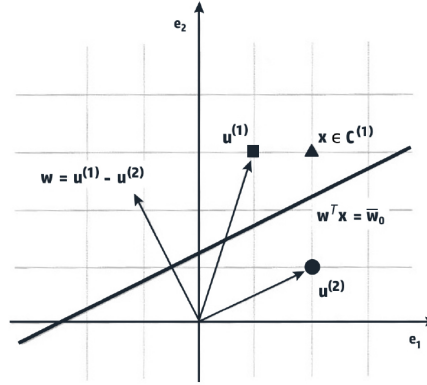
Ilustruje to Rysunek 1. Klasy $C^{(1)}$, $C^{(2)}$ są teraz określone przez relacje

$$C^{(1)} = \left\{ x \in \mathbb{R}^n : w^T x > \bar{w}_0, \right\}, \quad (11)$$

oraz

$$C^{(2)} = \left\{ x \in \mathbb{R}^n : w^T x < \bar{w}_0, \right\} \quad (12)$$

Ta obserwacja prowadzi do konstrukcji prostego urządzenia, które pobiera n -wymiarową daną $x \in \mathbb{R}^n$, oblicza iloczyn skalarny $w^T x$, a następnie sprawdza, czy jego wartość przekracza wartość progową $\bar{w}_0$, czy też nie. Pokazuje to schemat na Rysunku 2. Układ obliczający iloczyn skalarny oznaczony jest symbolem Σ , natomiast układ sprawdzający relację względem progu oznaczony jest symbolem UP . Te dwa elementy jako całość określane są nazwą *Jednostka liniowa z progiem* (w skrócie *LTU* - Linear Treshold Unit). LTU zwraca wartość zależnie od relacji iloczynu skalarnego względem progu. Jeżeli jego wartość przekracza wartość progową, to na wyjściu pojawia się wartość 1, a w przeciwnym przypadku wartość -1. Technicznie



Rysunek 1: Prosta rozdzielająca

wygląda to tak, że układ UP oblicza wartość funkcji progującej f na argumentie $w^T x$, przy czym funkcja progująca ma postać (lewy wykres na Rysunku 3)

$$f(h) = \begin{cases} -1 & \text{jeżeli } h \leq \bar{w}_0 \\ 1 & \text{jeżeli } h > \bar{w}_0 \end{cases} \quad (13)$$

Funkcja z układu UP jest w ogólności nazywana w teorii sztucznych sieci neuronowych *funkcją aktywacji* (pełna nazwa: *funkcja aktywacji sztucznego neuronu*), gdyż układ LTU jest pewnym modelem rzeczywistego neuronu w ludzkim mózgu.

Odnosząc się do wcześniej utworzonych dwóch klas, jeżeli dla pewnego $x \in \mathbb{R}^n$ LTU zwraca 1, to x należy do klasy $C^{(1)}$, a w przeciwnym przypadku należy do klasy $C^{(2)}$. Taki układ LTU można uogólnić tak, aby wartość progowa zawsze wynosiła 0.

Osiągane jest to poprzez dodanie do układu dodatkowego wejścia o stałej wartości 1, co oznacza, że dana wejściowa należy do przestrzeni cech $\mathbb{R}^{n+1}$. Zmodyfikowany układ przedstawiony jest na Rysunku 4. Układ Σ oblicza teraz iloczyn skalarny $\bar{w}^T \bar{x}$, gdzie

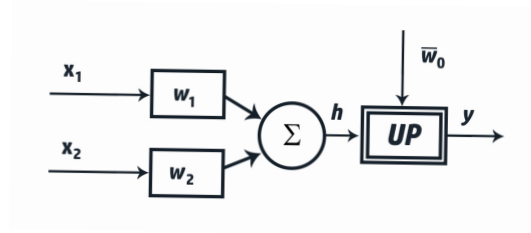
$$w^T = [-\bar{w}_0 \ w_1 \ \dots \ w_n], \quad x^T = [1 \ x_1 \ \dots \ x_n], \quad (14)$$

natomiast hiperpłaszczyzna rozdzielająca jest definiowana przez równanie

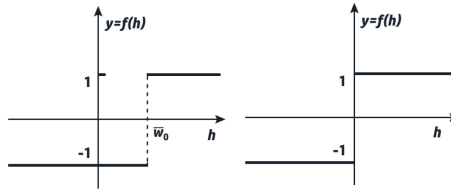
$$\bar{w}^T \bar{x} = 0. \quad (15)$$

Analogicznie wzorce $u^{(1)}$, $u^{(2)}$ przechodzą do rozszerzonej przestrzeni cech $\mathbb{R}^{n+1}$ i przybierają postać:

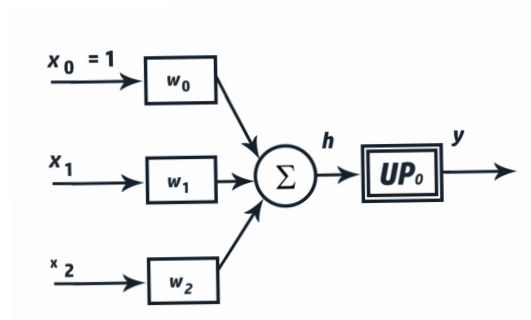
$$\bar{u}^{(1)} = \begin{bmatrix} 1 \\ u^{(1)} \end{bmatrix} = \begin{bmatrix} 1 \\ u_1^{(1)} \\ \vdots \\ u_n^{(1)} \end{bmatrix} \quad \bar{u}^{(2)} = \begin{bmatrix} 1 \\ u^{(2)} \end{bmatrix} = \begin{bmatrix} 1 \\ u_1^{(2)} \\ \vdots \\ u_n^{(2)} \end{bmatrix} \quad (16)$$



Rysunek 2: Układ klasyfikujący (n=2)



Rysunek 3: Charakterystyka (funkcja przejścia) układu progowego $L=UP$, $P=UP_0$ z progiem zerowym



Rysunek 4: Układ klasyfikujący z układem progowym UP_0 z progiem zerowym

Układ progujący (w tym przypadku oznaczony przez UP_0) posługuje się funkcją aktywacji f postaci (prawy wykres na Rysunku 3)

$$f(h) = \begin{cases} -1 & \text{jeżeli } h \leq 0 \\ 1 & \text{jeżeli } h > 0 \end{cases} \quad (17)$$

Istotne jest aby pamiętać, że działanie uzyskanego w ten sposób klasyfikatora ma sens tylko w przypadku, kiedy pierwsza współrzędna wektora $\bar{x}$ wynosi 1.

2 Perceptron

Poprzednia sekcja pokazała, że szukana hiperpłaszczyzna jest w ogólności reprezentowana przez równanie (15) (tzn. należy do grupy hiperpłaszczyzn przechodzących przez zero układu współrzędnych), natomiast funkcję progową opisuje równanie (17) (tzn. wartością progową jest zero). Klasyfikator jest zatem opisany przez funkcję

$$f(\bar{w}^T \bar{x}). \quad (18)$$

W ogólności funkcja progująca może mieć różną postać, więc przyjmijmy, że powyższą jej postać będziemy oznaczali przez $sign()$ (signum - funkcja znaku) - jest to standardowe w matematyce oznaczenie dla tej funkcji. Przyjmijmy teraz dodatkowo, że posiadamy więcej niż pojedyncze reprezentantów dwóch rozpatrywanych klas. Jasne jest, że wówczas znalezienie hiperpłaszczyzny granicznej musi polegać w ogólności na rozwiązaniu układu nierówności i może w pewnych przypadkach oznaczać brak takiej hiperpłaszczyzny. Spróbujmy sformułować rozwiązanie tego zagadnienia w postaci konstrukcji algorytmu znajdującego szukaną hiperpłaszczyznę. Przyjmijmy jednak, że algorytm znajdzie rozwiązanie jedynie w przypadku, gdy ono rzeczywiście istnieje (ale jest nieznane). Zatem przyjmując $\mathbb{R}^n$ jako przestrzeń cech oraz zmieniając oznaczenie wektora w na θ szukamy klasyfikatora postaci

$$y = sign(\bar{\theta}^T \bar{x}), \text{ gdzie } \bar{x}, \bar{\theta} \in \mathbb{R}^{n+1}. \quad (19)$$

Rozpocznijmy od precyzyjnego ustalenia postaci algorytmu. Przyjmijmy, że mamy w sumie m reprezentantów obu klas - oznaczamy ich przez $x_1, x_2, \dots, x_m$ (m_1 dla pierwszej klasy oraz m_2 dla drugiej, $m = m_1 + m_2$) oraz, że przynależność tych reprezentantów do obu klas jest oznaczona przez wartości (etykiety) $y_1, y_2, \dots, y_m$ z zbioru $\{-1, 1\}$ (tzn. w języku funkcji klasyfikatora). Zbiór oznaczonych reprezentantów klas nazywamy *zbiorem treningowym*, dlatego że będzie służył do algorytmowi do znalezienia poprawnej wartości wektora θ - wartość tą oznaczamy przez θ^* . Przyjmijmy również, że budujemy algorytm krokowego dochodzenia do poprawnej wartości wektora θ - jego postać w k -tym kroku oznaczamy przez $\theta^{(k)}$ ($k=0, 1, \dots$) - przyjmujemy, że $\theta^{(0)} = 0$ (wektor złożony z samych zer). W k -tym kroku algorytm wybiera (losowo, z powtórzeniami) jednego z reprezentantów x_i ze zbioru treningowego i wylicza nową wartość wektora $\theta^{(k)}$ posługując się formułą (i_k oznacza indeks wylosowanego w k -tym kroku elementu zbioru treningowego):

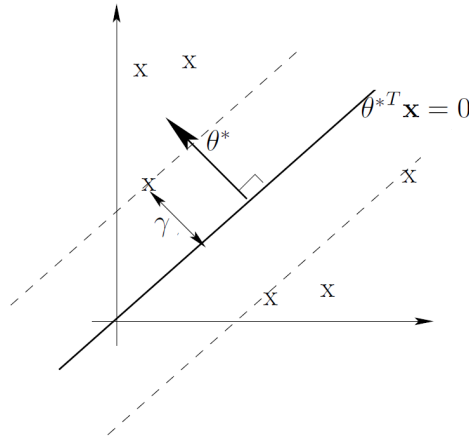
$$\theta^{(k)} = \begin{cases} \theta^{(k-1)} + \eta y_{i_k} x_{i_k} & \text{gdy } y_{i_k} (\theta^{(k-1)})^T x_{i_k} < 0, \\ \theta^{(k-1)} & \text{w przeciwnym przypadku} \end{cases}, \quad (20)$$

gdzie η jest *stałą szybkości uczenia*. Wynika z tego, że w k -tym kroku wartość $\theta^{(k)}$ zmienia się tylko w przypadku, gdy wylosowany element zbioru treningowego znajduje się po niewłaściwej stronie bieżącej postaci hiperpłaszczyzny granicznej (określonej w tym momencie całkowicie przez $\theta^{(k-1)}$). Przyjmujemy dodatkowo, że zbiór

treningowy jest wspólnie ograniczony, tzn. że istnieje $R > 0$ takie, że dla każdego $i = 1, 2, \dots, n$ $\|x_i\| \leq R$ - to założenie jest zupełnie naturalne dla danych z realnego świata. Ponadto przyjmujemy, że szukany klasyfikator rzeczywiście istnieje oraz, że spełnia następujący warunek:

$$\text{istnieje } \gamma > 0 \quad y_i(\theta^*)^T x_i > \gamma \quad \text{takie, że dla każdego } i = 1, 2, \dots, m \quad (21)$$

Oznacza to, że zakładamy dość dobrą jakość klasyfikatora: wszystkie elementy treningowe znajdują się poza pasem o szerokości 2γ i osi wyznaczonej przez graniczną hiperpłaszczyznę. Ilustruje to Rysunek 5. Udowodnimy teraz, że taki algorytm jest



Rysunek 5: Margines wokół hiperpłaszczyzny granicznej

zbieżny do wartości θ^* po skończonej ilości kroków.

Najpierw pokażemy, że iloczyn skalarny z elementami θ^* oraz $\theta^{(k)}$ rośnie co najmniej liniowo w każdym kroku. Z warunku (21) dostajemy, że dla każdego $i = 1, 2, \dots, m$

$$(\theta^*)^T \theta^{(k)} = (\theta^*)^T \theta^{(k-1)} + y_i (\theta^*)^T x_i > (\theta^*)^T \theta^{(k-1)} + \gamma. \quad (22)$$

Zatem po k krokach

$$(\theta^*)^T \theta^{(k)} > k\gamma. \quad (23)$$

Następnie pokażemy, jak zmienia się norma elementu $\theta^{(k)}$ w kolejnych krokach.

$$\begin{aligned} \|\theta^{(k)}\|^2 &= \|\theta^{(k-1)} + y_{i_k} x_{i_k}\|^2 \\ &= \|\theta^{(k-1)}\|^2 + 2y_{i_k} (\theta^{(k-1)})^T x_{i_k} + \|x_{i_k}\|^2 \\ &= \|\theta^{(k-1)}\|^2 + \|x_{i_k}\|^2 \\ &= \|\theta^{(k-1)}\|^2 + R^2 \end{aligned} \quad (24)$$

Przejście z linii drugiej do trzeciej wykorzystuje fakt, że zmiana wartości $\theta^{(k)}$ następuje tylko wtedy, gdy $y_{i_k}(\theta^{(k-1)})^T x_{i_k} < 0$, natomiast przejście z linii trzeciej do czwartej opiera się na założeniu wspólnej ograniczoności zbioru treningowego. Zatem

$$\|\theta^{(k)}\|^2 \leq kR^2. \quad (25)$$

Z równań (23) oraz (25) dostajemy

$$1 \leq \cos(\theta^*, \theta^{(k)}) = \frac{(\theta^*)^T \theta^{(k)}}{\|\theta^{(k)}\| \|\theta^*\|} \geq \frac{k\gamma}{\|\theta^{(k)}\| \|\theta^*\|} \geq \frac{k\gamma}{R\sqrt{k} \|\theta^*\|} \quad (26)$$

Stąd

$$k \leq \frac{R^2 \|\theta^*\|^2}{\gamma^2} \quad (27)$$

Skończoność wszystkich wartości po prawej stronie implikuje, że algorytm musi osiągnąć rozwiązanie po skończonej ilości kroków.

2.1 Perceptron z perspektywy teorii optymalizacji

Dla pewnej prostoty równanie hiperpłaszczyzny zapisujemy w postaci $\theta^T x - \theta_0 = 0$, czyli w formule (7). Ustalmy, że etykiety $y_i = 1$ odpowiadają półpłaszczyźnie o równaniu $\theta^T x - \theta_0 \geq 0$, natomiast etykiety $y_i = -1$ półpłaszczyźnie o równaniu $\theta^T x - \theta_0 < 0$. Wiadomo, że odległość punktu $x \in \mathbb{R}^n$ od hiperpłaszczyzny w $\mathbb{R}^n$ wynosi

$$\frac{|\theta^T x - \theta_0|}{\|\theta\|_2} \quad (28)$$

Jeżeli punkt x jest błędnie sklasyfikowany (otrzymujemy wartość ujemną funkcji $\text{sign}(\bar{\theta}^T \bar{x})$ dla punktu, który powinien leżeć powyżej prostej i wartość dodatnią dla punktu, który powinien leżeć poniżej prostej), to znakowana odległość (czyli wartość czasami ujemna) dla takiego punktu wynosi

$$\text{Err}_z(x) = -\frac{\theta^T x - \theta_0}{\|\theta\|_2} \quad (29)$$

Chcąc zachować dodaniem wartość odległości możemy wykorzystać jako mnożniki etykiety y_i punktów:

$$\text{Err}(x) = -y_i \frac{\theta^T x - \theta_0}{\|\theta\|_2} \quad (30)$$

Zatem suma po wszystkich źle sklasyfikowanych punktach (zbiór ich indeksów oznaczamy przez M) wyraża się formułą (współrzędne wektorów oznaczamy przez górne indeksy)

$$E(\theta, \theta_0) = -\frac{1}{\|\theta\|_2} \sum_{i \in M} y_i (\theta^T x_i - \theta_0) = -\frac{1}{\|\theta\|_2} \sum_{i \in M} y_i \left(\sum_{j=1}^n \theta^j x_i^j - \theta_0 \right) \quad (31)$$

Tę funkcję chcemy zminimalizować metodą największego spadku¹, więc rozpoczynamy od obliczenia gradientu funkcji $E(\theta, \theta_0)$. Wtedy otrzymujemy (wektor θ ma n współrzędnych)

$$\frac{\partial E}{\partial \theta^j}(\theta, \theta_0) = -\frac{1}{\|\theta\|_2} \sum_{i \in M} y_i x_i^j \quad (32)$$

$$\frac{\partial E}{\partial \theta_0}(\theta, \theta_0) = -\frac{1}{\|\theta\|_2} \sum_{i \in M} y_i \quad (33)$$

Zatem (gradient oznaczamy przez ∇)

$$\nabla E(\theta, \theta_0) = -\frac{1}{\|\theta\|_2} \left[\sum_{i \in M} y_i, \sum_{i \in M} y_i x_i^1, \sum_{i \in M} y_i x_i^2, \dots, \sum_{i \in M} y_i x_i^n \right]^T = -\frac{1}{\|\theta\|_2} \sum_{i \in M} y_i x_i \quad (34)$$

Stosując metodę stochastycznego największego spadku (w pojedynczym kroku korzystamy tylko z jednego punktu x_i) oraz włączając mnożnik $\frac{1}{\|\theta\|_2}$ do stałej szybkości uczenia (w takim przypadku zmienia się ona w każdym kroku) otrzymujemy wzór (20), co pokazuje, że perceptron jest w istocie algorytmem stochastycznego największego spadku.

Algorytm perceptronu posiada dwie główne wady:

- jeżeli dane są separowalne liniowo, to rozwiązanie istnieje, ale nie jest jednoznaczne (zależy od punktu startowego) i ponadto zbieżność może być wolna dla gęstego zbioru danych
- jeżeli dane nie są separowalne liniowo, to stosowanie algorytmu perceptronu może powodować pojawianie się trudnych do wykrycia długich cykli

$$\nabla E(\theta, \theta_0) = -\frac{1}{\|\theta\|_2} \left[\sum_{i \in M} y_i, \sum_{i \in M} y_i x_i^1, \sum_{i \in M} y_i x_i^2, \dots, \sum_{i \in M} y_i x_i^n \right]^T = -\frac{1}{\|\theta\|_2} \sum_{i \in M} y_i x_i \quad (35)$$

¹W języku teorii optymalizacji (również sieci neuronowych) funkcja, której minimum szukamy i która określa istotny błąd obliczeniowy, nazywana jest *funkcją kosztu* lub *funkcją straty*.